Block 4 – Session 2

Cloud Computing & Cloud Networking

Arquitectura de Xarxes

Universitat Pompeu Fabra

Adrián Pino <adrian.pino@upf.edu>

Outline

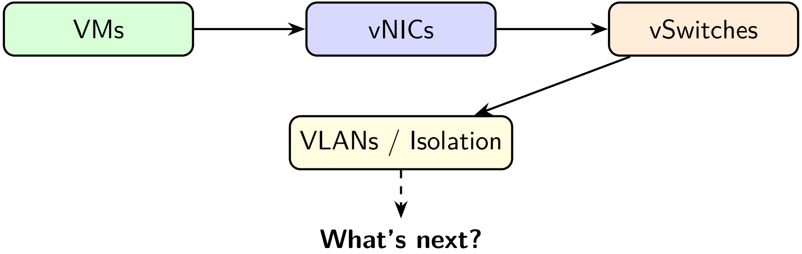
If We Can Virtualize Everything... Why Not Sell It?

Virtualization is the technology.
Cloud computing is the business model.

Learning objectives:

1. Understand cloud computing's definition and essential characteristics
2. Distinguish IaaS (Infrastructure as a Service), PaaS (Platform as a Service), and SaaS (Software as a Service) service models
3. Design a basic cloud virtual network with subnets and security controls

Recap – What We Virtualized (Session 1)



- Compute virtualization: VMs and containers
- Network virtualization: vNICs, vSwitches, vRouters, VXLAN overlays
- Together: the **building blocks** of the cloud

The Leap to Cloud

- Virtualization = technology (**how** you run multiple workloads)
- Cloud = business model (**what** you sell to customers)

Virtualization	**Cloud Computing**
Run multiple VMs on one host	Offer compute as a **service**
Internal IT optimization	External/internal consumption
Manual provisioning possible	**Automated, self-service**
You own the hardware	Provider owns the hardware

Key difference: **automation and self-service** (no phone calls, no tickets, no waiting).

Discussion: From Virtualization to Cloud

Your company already runs VMs on its own servers. Why would you move to the cloud instead of keeping everything in-house?

Hints: think about CapEx vs OpEx, scaling speed, and who manages what.

Outline

Cloud Computing – Definition (NIST: National Institute of Standards and Technology)

> *A model for enabling ubiquitous, convenient, **on-demand network access** to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort.*

Source: NIST SP 800-145 (2011). Still the standard definition used industry-wide.

Five Essential Characteristics

Characteristic	What it means
On-demand self-service	Provision resources without human interaction
Broad network access	Available over the network from any device
Resource pooling	Shared infrastructure, multi-tenant
Rapid elasticity	Scale up/down automatically
Measured service	Pay only for what you use (metered)

- **All five** must be present to qualify as "cloud computing"
- If you need to call someone to get a VM $\rightarrow$ it is not cloud

Source: NIST SP 800-145, Mell & Grance, 2011.

On-Demand Self-Service

- Users provision resources through a **portal or API (Application Programming Interface)**
- No phone calls, no tickets, no waiting for approval
- Example: launch a VM in **30 seconds** via a web console

Analogy: vending machine (cloud) vs restaurant with a waiter (traditional IT).

Broad Network Access + Resource Pooling

Broad network access:

- Resources accessible from **any device** with network connectivity
- Standard protocols: HTTPS, SSH, REST APIs
- No proprietary client software needed

Resource pooling:

- Provider's resources serve **multiple tenants** simultaneously
- Tenants do not know (or care) where their resources are physically located
- Dynamically assigned based on demand
- Builds on the **multi-tenancy** concepts from Session 1

Rapid Elasticity + Measured Service

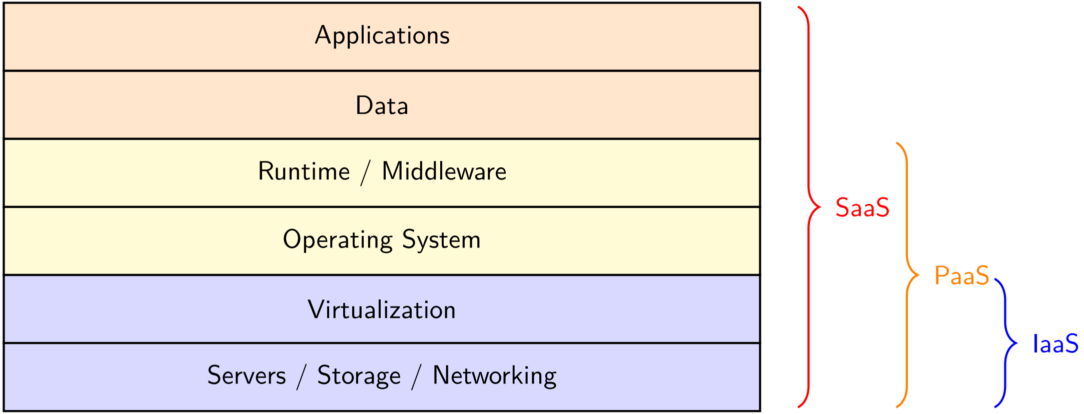
Rapid elasticity:

- Resources appear **unlimited** to the consumer
- Scale out (add) and scale in (remove) automatically
- Example: web store on Black Friday $\rightarrow$ $10\times$ servers, then back to normal

Measured service:

- Usage is **metered** (CPU-hours, GB stored, GB transferred)
- **Pay-per-use** model $\rightarrow$ OpEx instead of CapEx
- Transparent monitoring and billing reports

Cloud Service Models – The Stack



IaaS – Infrastructure as a Service

- Provider gives you: **VMs, storage, networks**
- You manage: OS, middleware, applications, data
- **Maximum control, maximum responsibility**

Examples:

- **AWS EC2 (Elastic Compute Cloud)**: virtual machines
- **Azure Virtual Machines**
- **Google Compute Engine**

Use case: "I want a Linux server in Frankfurt, 4 CPUs, 16 GB RAM, right now."

Source: AWS EC2 Documentation, docs.aws.amazon.com.

PaaS – Platform as a Service

- Provider gives you: **runtime environment + tools**
- You manage: only your **application code and data**
- No need to manage OS, patches, or scaling infrastructure

Examples:

- **Google App Engine**
- **AWS Elastic Beanstalk**
- **Heroku**

Use case: "I wrote a Python web app. Just run it, scale it, I do not care about servers."

SaaS – Software as a Service

- Provider gives you: a **complete application**
- You manage: your **data and user configuration**
- No installation, no maintenance, no updates to worry about

Examples:

- **Google Workspace** (Gmail, Docs, Drive)
- **Microsoft 365** (Word, Excel, Teams)
- **Salesforce**, **Slack**, **Zoom**

Use case: "I need email for 500 employees. I do not want to run a mail server."

Service Models – Who Manages What?

Component	IaaS	PaaS	SaaS
Applications	You	You	Provider
Data	You	You	You
Runtime / Middleware	You	Provider	Provider
Operating System	You	Provider	Provider
Virtualization	Provider	Provider	Provider
Hardware	Provider	Provider	Provider

- IaaS $\rightarrow$ SaaS: **less control, less responsibility**
- Major providers (AWS, Azure, GCP) offer all three models

Discussion: Cloud Fundamentals

A university wants to offer a web-based tool
for students to submit assignments.
Should they choose IaaS, PaaS, or SaaS? Why?

Hints: consider the IT team size, maintenance burden, and how much control they need.

Outline

Putting It All Together

- In Session 1 we saw the **building blocks**: vSwitches, vRouters, vNICs, VLANs
- These components do not run in isolation; they are managed by **platforms**
- Platforms orchestrate **VMs** or **containers** and configure virtual networking under the hood

Two categories:

- **Cloud platforms**: manage infrastructure at scale
  - Compute, storage, networking, and **VMs**
- **Container orchestration platforms**: deploy and scale **containerized** applications

Cloud Platforms

- **Self-managed / Private Cloud**: you own or rent the servers, install and manage the virtualization software yourself. Full control, but requires dedicated staff and expertise.

Platform	**Description**	**Logo**
OpenStack	Open-source, widely used in private clouds	
VMware vSphere	Enterprise leader, proprietary	
Proxmox VE	Open-source, lightweight alternative	

- **Managed / Public Cloud**: rent capacity on demand, pay-as-you-go. Provider handles hardware, networking, cooling, security, and updates.

Provider	**Description**	**Logo**
Amazon Web Services	Market leader since 2006	
Microsoft Azure	Strong enterprise integration	
Google Cloud Platform	Data and AI focus	

These map directly to the IaaS model we just covered: the provider (or you) manages compute, storage, and networking.

Container Orchestration Platforms

- **Self-hosted / Private**: you install and operate on your own servers

Platform	Description	Logo
Docker	Container runtime; works well on a single host Managing hundreds of containers across hosts? Unmanageable	docker
Kubernetes (K8s)	Created to solve this: scheduling, scaling, networking, self-healing	kubernetes
OpenShift	Enterprise K8s by Red Hat; adds security, CI/CD, UI	Red Hat OpenShift

- **Hosted / Public**: cloud providers offer the same platforms as managed services. Each provider uses different names for essentially the same product (e.g., AWS ECS/EKS, Azure AKS, Google GKE, OpenShift on all major clouds).

Discussion: Platforms

A startup with 5 engineers needs to deploy a web application that may go from 100 to 100,000 users. Would you build a private cloud or use a public one? Why?

Hints: team size, upfront cost, time to market, scaling needs, control over infrastructure.

Outline

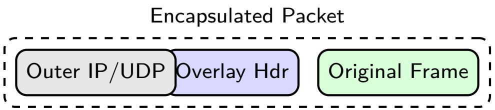
The VLAN Scalability Problem

- In Session 1 we used **VLANs** to isolate tenants on virtual networks
- VLANs use a **12-bit ID** → maximum **4,096** virtual networks
- Cloud providers host **millions** of tenants → VLANs are not enough
- Additional limitations:
  - VLANs are confined to a **single L2 domain**
  - Moving a VM to another host may require VLAN reconfiguration

We need a technology that scales beyond 4K networks and works across L3 boundaries.

Overlay Networks: Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: wrap virtual frames inside physical packets



- The physical network only sees the **outer headers**
- The virtual (tenant) traffic is hidden inside → **full isolation**

VXLAN: Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
  - 2^{24} = **16 million** virtual networks (vs 4,096 VLANs)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints)
  - Encapsulate at source, decapsulate at destination

Source: IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

VXLAN: How It Works

- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network does not need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology

This is the foundation of cloud networking. Next: how providers build **virtual networks** on top of overlays.

Source: IETF RFC 8926, “Geneve: Generic Network Virtualization Encapsulation,” 2020.

Discussion: Overlay Networks

Why can the physical network remain simple
if we use overlay networks?
What are the trade-offs of encapsulation?

Hints: think about overhead (extra headers), MTU implications, and troubleshooting complexity.

Outline

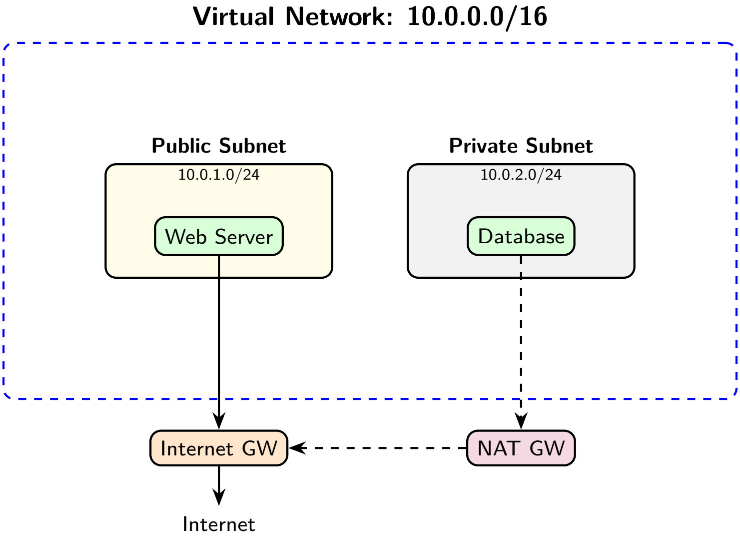
Virtual Networks in the Cloud

- Every cloud provider lets you create your own **isolated virtual network**
  - AWS: Virtual Private Cloud (VPC); Azure: Virtual Network (VNet); GCP: VPC Network
- Think of it as your **private data center network** in the cloud
- You define:
  - **Address space** (e.g., `10.0.0.0/16`)
  - **Subnets** (subdivisions of the address space)
  - **Routing rules** and **security policies**

Key: virtual networks are built on top of the **overlay network** concepts we just covered.

Source: AWS, "Amazon VPC User Guide"; Azure, "Virtual Network Documentation"; GCP, "VPC Documentation."

Virtual Network Architecture



Subnets – Public vs Private

Public subnet:

- Has a route to an **Internet Gateway**
- Instances can have **public IP addresses**
- Accessible from the Internet (if security rules allow)
- Use for: web servers, load balancers, bastion hosts

Private subnet:

- **No direct route** to the Internet
- Instances only have **private IPs**
- Can access Internet outbound via **NAT Gateway**
- Use for: databases, internal services, application backends

Route Tables

- Each subnet is associated with a **route table**
- Route table = set of rules determining where traffic goes

Destination	Target	Subnet type
`10.0.0.0/16`	local (within virtual network)	Both
`0.0.0.0/0`	Internet Gateway	Public
`0.0.0.0/0`	NAT Gateway	Private

- **Local route**: traffic within the virtual network stays inside (always present)
- **Default route** (`0.0.0.0/0`): where to send everything else
- Public subnets → IGW; Private subnets → NAT GW

Internet Gateway vs NAT Gateway

Internet Gateway (the front door):

- **Bidirectional**: the world can reach your instance, and it can reach the world
- Instance needs a **public IP**
- Use for: web servers, load balancers, anything public-facing

NAT Gateway (the fire exit):

- **One-way**: your instance can reach the Internet (e.g., download updates, call APIs), but nobody outside can reach it directly
- Instance keeps a **private IP** only
- Use for: databases, backends that need to download patches but must stay hidden

Same **NAT concept from Block 3**, now as a managed cloud service.

Security Groups – Instance-Level Firewall

- **Security group** = virtual firewall attached to an instance
- Rules specify allowed **inbound** and **outbound** traffic
- **Stateful**: allow inbound → response automatically allowed

Direction	**Protocol**	**Port**	**Source/Dest**
Inbound	TCP	80	`0.0.0.0/0`
Inbound	TCP	443	`0.0.0.0/0`
Inbound	TCP	22	`10.0.0.0/16`
Outbound	All	All	`0.0.0.0/0`

Source: AWS, "Security Groups for Your VPC," docs.aws.amazon.com.

Network ACLs (Access Control Lists) – Subnet-Level Firewall

- **Network ACL (Access Control List)** = firewall at the **subnet** level
- Rules evaluated in **order** (numbered, first match wins)
- **Stateless**: must explicitly allow both inbound AND outbound

Rule #	Direction	Protocol	Port	Action
100	Inbound	TCP	80	ALLOW
200	Inbound	TCP	443	ALLOW
*	Inbound	All	All	DENY

Source: AWS, "Network ACLs," docs.aws.amazon.com.

Security Groups vs Network ACLs

	Security Groups	Network ACLs
Scope	Instance level	Subnet level
State	Stateful	Stateless
Rules	Allow only	Allow + Deny
Evaluation	All rules evaluated	First match wins
Default	Deny all inbound	Allow all

Defense in depth: use both together.

- Security Groups → fine-grained per-instance control
- Network ACLs → broad subnet-level guardrails

Discussion: Cloud Networking

You deploy a web application in your cloud virtual network. The web server needs to be public, but the database must not be reachable from the Internet. How?

Hints: think about public vs private subnets, security groups, and NAT gateways.

Outline

Key Takeaways

1. Virtualization is the **technology**; cloud is the **business model**
2. Five NIST characteristics define true cloud computing
3. **IaaS / PaaS / SaaS** differ in who manages what
4. **VXLAN** overlays scale isolation to 16M networks (vs 4K VLANs)
5. A **virtual network** is your isolated cloud network, built on top of overlays
6. **Security groups** (instance) + **ACLs** (subnet) = defense in depth

Discussion

You are designing a cloud network for a company with public web servers and private databases. Which components would you use and why?

Think about: public vs private subnets, security groups, NAT gateways, route tables.

References

1. NIST SP 800-145, Mell & Grance, “The NIST Definition of Cloud Computing,” 2011.
2. NIST SP 800-144, “Guidelines on Security and Privacy in Public Cloud Computing,” 2011.
3. AWS, “Amazon VPC User Guide,” docs.aws.amazon.com.
4. Microsoft Azure, “Azure Virtual Network Documentation,” docs.microsoft.com.
5. Google Cloud, “VPC Documentation,” cloud.google.com/vpc/docs.
6. CSA, “Security Guidance for Critical Areas of Focus in Cloud Computing v5,” 2022.
7. Erl, Puttini & Mahmood, *Cloud Computing: Concepts, Technology & Architecture*, Prentice Hall, 2013.
8. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
9. IETF RFC 7348, “Virtual eXtensible Local Area Network (VXLAN),” 2014.
10. IETF RFC 8926, “Geneve: Generic Network Virtualization Encapsulation,” 2020.